

TITLE PLACEHOLDER

Anonymous Author(s)

Abstract

ABSTRACT PLACEHOLDER — write last, per PAPER_PLAN.md Phase 3.7.

1 Introduction

Embedding a small, reconfigurable fabric into an ASIC to implement one security-critical IP block — so that the function is not recoverable from the silicon layout or GDSII without the bitstream — is a known defense against reverse engineering and IP theft [?]. A generic, general-purpose FPGA architecture is built to serve arbitrary circuits, and is therefore over-provisioned — extra LUTs, routing, and DSP/BRAM capacity — for any one fixed design that will only ever run a single bitstream. Because the eFPGA masking use case targets exactly one known application on a small embedded fabric, rather than general-purpose reconfigurability, a custom-tailored architecture and placement, sized and arranged specifically for that one netlist, is viable, and can recover much of the area/delay/power overhead a generic FPGA would otherwise impose. We use the product of area, delay, and power (ADP) as our optimization target throughout this paper because it is precisely the size of this overhead — the “masking tax” of moving a protected block onto reconfigurable fabric instead of hardening it directly into standard cells.

Two prior papers establish that per-instance search over DSP/BRAM placement (and, in one case, CLB/arithmetic-block sizing) can substantially reduce this tax: GOLDS [?] uses a genetic algorithm with an Area×Delay fitness function and reports up to ~35% ADP improvement on the `diffeq1` benchmark; a follow-up applies NSGA-II to a larger, multi-objective search space that additionally co-evolves CLB LUT size and custom arithmetic-block sizing [?]. Both, however, are *testbench-specific*: the genetic algorithm’s population is re-initialized and re-evolved from scratch for every new protected IP block, and every generation of every run requires its own full, expensive VTR evaluation. Nothing learned while optimizing one netlist’s layout carries over to the next netlist — each new target benchmark pays the full search cost again, with no way to amortize the work already done on previous designs.

This paper asks whether a *learned policy* can replace that per-instance search loop, rather than just compete with it once per benchmark. RL placing blocks onto an *already-fixed* FPGA fabric is not new; what we ask is whether the same RL machinery can instead be used to *construct* the custom fabric layout itself — choosing DSP/BRAM placement and aspect ratio for a small, embedded FPGA built around one specific netlist, exactly the regime in which a per-instance, testbench-fixed GA has so far been the only approach. Concretely, we pose two research questions:

- **RQ1.** Reinforcement learning has been used to lay out blocks on a *fixed* FPGA fabric; can the same RL formulation instead be used to determine the fabric’s custom layout itself, for a given netlist — the embedded-FPGA logic-masking setting where a purpose-built, netlist-specific architecture is viable in the first place?

- **RQ2.** Can such an RL agent be *benchmark-agnostic* — pre-trained across a diverse pool of netlists so that the same checkpoint speeds up layout construction for a genuinely new, previously unseen custom benchmark, either directly (zero-shot) or after a short fine-tuning pass, rather than requiring a fresh search from scratch for every new protected IP block the way GOLDS and the NSGA-II follow-up do?

We train a single reinforcement-learning policy, conditioned on a graph-neural-network encoding of the target netlist, to place DSP/BRAM hard blocks and choose a fabric aspect ratio, evaluated directly against the real VTR (Yosys + VPR) synthesis-to-routing flow rather than a learned or analytic proxy. Because the same checkpoint can be applied to a netlist it has never seen, without re-running any search, this amortizes the cost of finding a good layout across deployments — the practical advantage we are after is not necessarily winning every single-instance ADP comparison outright, but being fast and strong on a genuinely new protected IP block.

We report three results. First, on a controlled, fully apples-to-apples comparison — our own genetic algorithm and our RL policy evaluated under the *identical* oracle, ADP metric, and traditional baseline, on a benchmark held out of RL training — RL reaches 74.06% ADP reduction versus the GA’s 68.17% (Section ??). Second, jointly trained across 11 benchmarks, the policy achieves a 35.7% aggregate in-pool ADP reduction, with 10 of 11 benchmarks finishing positive and one persistent laggard (Section ??). Third, recomputed on GOLDS’ own published 2-term Area×Delay metric and baseline architecture, a zero-shot evaluation — a policy trained only on `diffeq2`, applied directly to `diffeq1` with no benchmark-specific training at all — reaches 37.4% versus GOLDS’ published ~35% (Section ??). We also report a less convenient finding alongside these: across two seeds, zero-shot generalization to six held-out benchmarks is reasonably stable in aggregate (31.3% vs. 30.3% average ADP reduction) but with real per-benchmark swings of up to 13 points (Section ??); we present this honestly as a characterization of the current approach’s reliability rather than smoothing it into a single point estimate.

This paper sits at the intersection of two lines of work that have not, to our knowledge, been connected before (Section ??): custom eFPGA layout search for logic masking, which to date has used only per-instance evolutionary search [? ?], and learned policies for physical design, which to date have targeted ASIC macro placement at a much larger scale and through a learned reward proxy rather than the true toolchain [? ?]. We bring the amortization argument from the latter line of work to the former problem, evaluated end-to-end against the real VTR oracle the former line of work already established as the right fidelity bar for this problem.

Concretely, this paper contributes:

- A reinforcement-learning formulation of custom eFPGA DSP/BRAM placement and aspect-ratio selection, with a GCN-based netlist encoding (Section ??) and a fixed-size policy that remains loadable against benchmarks never seen

during training (Section ??), evaluated directly against the same VTR-based oracle used by prior eFPGA-layout search work.

- A fully controlled, apples-to-apples comparison against a from-scratch genetic algorithm under an identical evaluation oracle, ADP metric, and traditional baseline, on a benchmark held out of RL training: 74.06% ADP reduction for RL versus 68.17% for the GA (Section ??).
- A joint 11-benchmark training result (35.7% aggregate in-pool ADP reduction, 10 of 11 benchmarks positive) (Section ??).
- A literature-facing comparison against GOLDS’ own published metric and baseline architecture, recomputed precisely on a genuine zero-shot evaluation with no benchmark-specific training at all: 37.4% versus GOLDS’ published ~35% (Section ??).
- A two-seed characterization of zero-shot generalization reliability on six held-out benchmarks — stable in aggregate (31.3% vs. 30.3% average ADP reduction across seeds) but with real per-benchmark swings of up to 13 points — reported as a finding about the current approach’s limits, not smoothed into a single optimistic point estimate (Section ??).

2 Background and Related Work

2.1 eFPGA-Based Logic Masking

Embedding a reconfigurable fabric into an ASIC to hide one IP block’s function behind a bitstream is a hardware-security technique distinct from standard logic locking or camouflaging, since the protected function is not merely obfuscated in the netlist but physically absent from the fixed silicon until configured. GOLDS [?] introduced custom-tailored eFPGA layout design for this purpose: a genetic algorithm searches over DSP/BRAM tile placement on a fixed-size fabric, with an Area×Delay fitness function evaluated through the real VTR flow, and reports up to ~35% ADP-relevant improvement over a traditional island-style layout on diffeq1. A follow-up [?] extends this to a multi-objective NSGA-II search that additionally co-evolves CLB LUT size and custom DSP/BRAM block sizing — not just their placement — optimizing delay and power jointly and reporting up to ~44.6% improvement on a larger design-space. Both papers’ searches are run fresh, from a random population, for every new protected design; neither carries information learned on one IP block forward to the next. This paper targets the same problem GOLDS defines (custom DSP/BRAM placement plus aspect ratio, evaluated through the same VTR-based oracle) and treats it as the narrower, more directly comparable target for our controlled experiments (Section ??), while citing the NSGA-II extension as context for the larger design space a future version of this approach could also cover (Section ??).

2.2 Learned Policies for Physical Design

Reinforcement learning has been applied to chip-scale physical design problems structurally similar to ours. AlphaChip [?] trains an RL agent to place macro blocks for ASIC floorplanning, using a learned reward proxy in place of a full place-and-route evaluation per step, and reports that the resulting policy transfers across chip

designs without retraining from scratch — the same amortization argument we make for eFPGA placement, applied to a different physical-design problem and at a much larger design scale (full TPU floorplans rather than DSP/BRAM tile placement on a small embedded fabric). ChiPFormer [?] instead poses macro placement as an offline RL problem over a transformer policy, trained on a fixed dataset of prior placements rather than online environment interaction. Both works motivate the same underlying claim this paper tests in a different, smaller-scale setting: that a trained policy can out-amortize per-instance search once a population of designs needs to be placed, rather than re-running search from scratch on each one. Unlike AlphaChip, we evaluate every candidate placement against the true VTR synthesis-to-routing flow rather than a learned proxy, trading evaluation speed for evaluation fidelity — appropriate at our problem’s much smaller scale, where a full VTR run completes in seconds to low minutes (Section ??).

2.3 VTR/VPD Background

We build directly on the open-source Verilog-to-Routing (VTR) [?] toolchain: Yosys performs technology-independent synthesis, and VPR performs packing, placement, and routing against an architecture described in an XML schema specifying tile types, their positions on the fabric, and their connectivity. This architecture XML is the same artifact GOLDS and the NSGA-II follow-up generate by hand-coded search and that we generate by policy rollout (Section ??); VTR’s willingness to accept an essentially arbitrary heterogeneous tile layout, rather than only the regular island-style grids it ships by default, is what makes the entire custom-eFPGA-layout line of work — GOLDS, the NSGA-II follow-up, and this paper — possible in the first place.

3 Methodology

3.1 Problem Formulation

Given a fixed eFPGA fabric template parameterized by a tile grid of width W and height H , and a netlist requiring n_{dsp} DSP and n_{bram} BRAM hard blocks, our goal is to choose a placement $\pi = (\text{ar}, \{(x_i, y_i)\}_{i=1}^{n_{\text{dsp}}+n_{\text{bram}}})$ — an aspect ratio $\text{ar} \in \mathcal{R}$ for the fabric plus a tile coordinate for every hard block — that minimizes the post-synthesis, post-place-and-route cost of implementing that netlist on the resulting custom architecture. We use the product of routing area, critical-path delay, and power (ADP) as the cost, since this directly reflects the silicon and timing overhead of moving the protected IP block onto reconfigurable fabric:

$$\text{ADP}(\pi) = \text{Area}(\pi) \times \text{Delay}(\pi) \times \text{Power}(\pi), \quad (1)$$

where each term is obtained by running the candidate placement through the full VTR (Yosys + VPR) synthesis-to-routing flow (Section ??). We report results as percentage ADP reduction relative to a traditional, non-customized island-style eFPGA baseline at the same logic capacity.

3.2 RL Formulation

We pose placement as a finite-horizon sequential decision process and train a policy with Proximal Policy Optimization (PPO), using the maskable variant (sb3_contrib’s MaskablePPO) so that the

action space can be restricted to currently-legal tile coordinates at every step.

Episode structure. An episode places exactly one benchmark’s blocks onto the shared canvas. Step 0 selects the fabric aspect ratio from a fixed discrete set $\mathcal{R}$; subsequent steps place each DSP, then each BRAM instance, one per step, in descending order of shared-net connectivity. Once every block is placed, the environment bakes the resulting layout into an architecture XML and constraints file (via Jinja2 templating) and invokes the VTR flow to obtain $\text{Area}(\pi)$, $\text{Delay}(\pi)$, and $\text{Power}(\pi)$; all intermediate steps receive zero reward.

Action space. The action space is $\text{Discrete}(W_{\max} \times H_{\max})$, fixed across all benchmarks in a training run (Section ??). At step 0 an action index selects an entry in $\mathcal{R}$; at later steps an action index decodes to a tile coordinate (x, y) via a fixed stride on $H_{\max}$, independent of which benchmark is active. Action masking — keyed off the *active* benchmark’s real width/height and the blocks already placed — restricts the policy to legal, non-overlapping, in-bounds coordinates for that specific benchmark and step.

Reward. The terminal reward is a weighted sum of negative log-ratios against the traditional baseline for the active benchmark:

$$r = -\left(w_a \log \frac{\text{Area}}{\text{Area}_0} + w_d \log \frac{\text{Delay}}{\text{Delay}_0} + w_p \log \frac{\text{Power}}{\text{Power}_0} + w_w \log \frac{\text{WL}}{\text{WL}_0}\right), \quad (2)$$

where $\text{Area}_0, \text{Delay}_0, \text{Power}_0, \text{WL}_0$ are the traditional-baseline metrics for that benchmark and w_a, w_d, w_p, w_w are configurable weights (Section ??). When $w_a = w_d = w_p = 1$ and $w_w = 0$, r reduces to $\log(\text{ADP}_0/\text{ADP})$ — i.e. the reward is the log of the ADP-improvement ratio in Eq. ??, making reward and our headline metric directly comparable. An invalid choice (degenerate aspect ratio, out-of-bounds or overlapping placement, or a VTR flow failure) terminates the episode early with a fixed penalty reward of -10 .

3.3 GCN Feature Extractor

To let a single policy generalize across benchmarks with different netlist sizes, hard-block counts, and grid dimensions, we first reduce each benchmark’s full netlist graph to a compact, fixed-schema representation: one node per DSP/BRAM hard-block instance, plus one aggregate “fabric” node summarizing the remaining standard-cell logic. Each node carries a 4-dimensional feature vector $[\text{is_dsp}, \text{is_bram}, \text{is_fabric}, \text{net_count_normalized}]$. Edges between two hard-block nodes are weighted by their raw shared-net count; edges between a hard-block node and the fabric node are weighted by the summed shared-net count across all standard-cell members that block touches. Because these two edge kinds live on very different numeric scales, we normalize each into $[0, 1]$ separately with a log compression, $w_{\text{norm}} = \log(1 + w)/\log(1 + c)$, using a per-kind ceiling c calibrated to the observed maximum across the training benchmark pool — a single shared linear ceiling either saturates the larger-magnitude fabric edges or crushes the direct edges near zero.

This reduced graph is padded to a fixed $(\text{MAX_NODES}, \text{MAX_EDGES})$ — computed once over the full benchmark *universe* (Section ??) — and fed to a small GNN feature extractor shared by the policy and value heads: two GCNConv layers (hidden width 64) produce a per-node embedding h_i ; a global mean pool over h gives a whole-netlist embedding, and the embedding of the specific node being placed

this step (looked up via `current_block_idx`) gives a focused, per-step embedding. These are concatenated with (i) a small CNN over the occupancy grid (4 channels: occupancy, block-type hint, aspect ratio, and net count, each padded with an out-of-canvas sentinel outside the active benchmark’s real footprint) and (ii) the active benchmark’s normalized (width, height), then projected to a 128-dimensional joint feature vector consumed by the PPO policy and value heads. The whole extractor is trained end-to-end under the standard PPO loss; there is no separate pretraining stage for the GNN.

3.4 Universe Superset and Action Masking

A policy network’s input/output dimensions are fixed at construction time, but we want one checkpoint to remain loadable against benchmarks it was never trained on, including ones discovered after training. We address this by sizing every dimension-dependent quantity — canvas $(W_{\max}, H_{\max})$, `MAX_NODES`, `MAX_EDGES` — from a benchmark *universe* that is a deliberate superset of the benchmarks actually sampled during training: it can include benchmarks reserved purely for zero-shot evaluation, so long as none of them exceeds the resulting shared dimensions. Per-episode action masking (Section ??) and the `valid_wh` observation are what let the same fixed-size network operate correctly on each benchmark’s much smaller real footprint within that shared canvas.

3.5 Training Setup

4 Experimental Setup

4.1 Evaluation Oracle

Every candidate placement is scored by the true downstream toolchain, not a proxy. A placement (aspect ratio + hard-block coordinates) is rendered into a complete VTR architecture XML and a VPR placement-constraints file via Jinja2 templates parameterized by tile coordinates, block type, and fabric aspect ratio. This architecture and the benchmark’s Verilog are then run through the full VTR flow — Yosys-based synthesis, VPR packing, placement, and routing, followed by VPR’s power estimator against a 45nm PTM technology file — and we extract routing area, critical-path delay, dynamic power, and wirelength from the resulting VPR reports. Evaluated (placement, benchmark) pairs are cached, since the synthesis-to-routing flow is the dominant cost of both training and evaluation (seconds to low minutes per call depending on benchmark size).

4.2 Baseline

Our traditional baseline is VTR’s default island-style architecture template (`k6_frac_N10_mem32K_40nm`: 6-input LUTs, cluster size 10, 32K-bit BRAMs, 40nm) at matched logic capacity for each benchmark — the same template used by `GOLDS [?]` as its traditional-layout comparison point, which lets us compare percentage improvements directly against theirs on a shared starting point (Section ??).

4.3 Benchmarks

Table ?? lists the benchmark pool used in this work: an 11-benchmark training set of real circuits spanning a range of DSP/BRAM resource

Table 1: Benchmark pool: grid size and hard-block requirements.

Benchmark	Grid	DSP	BRAM
<i>Training set (11 benchmarks)</i>			
fifo	6×6	0	1
ch_intrinsics	10×10	0	1
diffeq2	14×14	5	0
boundtop	13×13	0	1
diffeq1	14×14	5	0
spree	12×12	1	3
mkSMAdapter4B	18×18	0	5
or1200	25×25	1	2
mmc_core	13×13	0	1
mkPktMerge	26×26	0	15
raygentop	17×17	6	1
<i>Held-out (zero-shot only)</i>			
softmax	41×41	8	0
reduction_layer	42×42	0	32
lightweight_cipher	12×12	0	3
custom_macbuf	12×12	3	1
mkDelayWorker32B	48×48	0	43
arm_core	35×35	0	24

profiles, and a held-out set used only for zero-shot evaluation (Section ??), never sampled during training.

4.4 Baselines Compared

We compare against: (1) the traditional, non-customized island-style eFPGA baseline (Section ??); (2) a from-scratch genetic algorithm we implemented under the identical evaluation oracle and ADP metric, giving a fully apples-to-apples in-house comparison (Section ??); and (3) prior published results from GOLDS [?] and the NSGA-II-based follow-up [?], subject to the metric caveat in Section ??.

5 Results

5.1 In-Pool Performance

Table ?? reports per-benchmark ADP reduction for the 11-benchmark policy (seed 42, 13,000 episodes) directly on its own training set. Aggregate ADP reduction — computed as one minus the ratio of summed ADP across all 11 benchmarks to summed baseline ADP, *not* a simple mean of per-benchmark percentages — is **35.73%**, with 10 of 11 benchmarks finishing positive. raygentop is the one persistent laggard (−2.92%), which we attribute tentatively to its 6-DSP/1-BRAM profile being a thin slice of the joint training distribution, and to it being the slowest-converging benchmark in this pool, rather than a fundamental failure mode. A second seed (seed 7, same configuration, also 13,000 episodes) reaches a lower pooled aggregate of 23.11%, driven mostly by or1200 and boundtop finishing well below their seed-42 result — we discuss this seed-to-seed spread alongside the zero-shot results in Section ?? rather than treating either seed’s in-pool number in isolation as the policy’s true performance.

Table 2: In-pool ADP reduction, 11-benchmark policy, seed 42 (seed 7 in parentheses).

Benchmark	ADP reduction
diffeq2	+68.2% (+68.3%)
fifo	+60.2% (+60.2%)
diffeq1	+57.4% (+48.5%)
boundtop	+50.7% (+33.8%)
mkPktMerge	+40.2% (+40.2%)
spree	+38.6% (+39.1%)
ch_intrinsics	+36.3% (+36.3%)
mkSMAdapter4B	+36.2% (+34.9%)
or1200	+33.5% (+5.8%)
mmc_core	+25.2% (+24.4%)
raygentop	−2.9% (−2.1%)
Aggregate (pooled)	+35.73% (+23.11%)

Table 3: GA vs. RL head-to-head on custom_macbuf (baseline ADP = 44,503).

Method	Best ADP	Reduction
GA (500-gen cap, patience 50)	14,164.5	68.17%
RL (fine-tuned, 11-bench policy)	11,553.2	74.06%

5.2 GA Head-to-Head Comparisons

To establish a fully controlled, apples-to-apples comparison against evolutionary search — not subject to any metric or baseline-matching caveat — we implemented our own genetic algorithm under the identical evaluation oracle, ADP metric, and traditional baseline as our RL pipeline, and ran it on custom_macbuf (held out of RL training; Table ??): roulette-wheel selection, single-point crossover, per-gene mutation over both hard-block coordinates and fabric aspect ratio, population 5, capped at 500 generations with an early-stop patience of 50 generations without improvement (converged at generation 186). The RL comparison point is the 11-benchmark policy (Section ??) fine-tuned on custom_macbuf alone. Table ?? reports the result: RL reaches **74.06%** ADP reduction versus the GA’s **68.17%** on the identical benchmark, metric, and toolchain.

5.3 Head-to-Head Against GOLDS

As a secondary, literature-facing comparison, we recompute a genuine zero-shot result — a policy trained only on diffeq2, evaluated zero-shot on diffeq1 with no training on it whatsoever — on GOLDS’ [?] own 2-term Area×Delay metric (rather than our 3-term Area×Delay×Power metric used elsewhere), under the same traditional baseline architecture GOLDS uses. This yields a **37.4%** reduction versus GOLDS’ published best-of-population result of **~35%** on the same benchmark — a real, if narrower, margin, achieved with zero benchmark-specific training. We report this result separately from Section ?? precisely because it required recomputing on a different metric than the rest of this paper; nowhere else do we compare 2-term GA numbers against our 3-term RL numbers directly.

Table 4: Zero-shot ADP reduction on six held-out benchmarks, 11-benchmark policy.

Benchmark	Seed 42	Seed 7
mkDelayWorker32B	+56.7%	+62.4%
custom_macbuf	+61.2%	+47.8%
lightweight_cipher	+30.7%	+30.2%
reduction_layer	+21.7%	+28.9%
arm_core	+12.1%	+7.0%
softmax	+5.4%	+5.6%
Average	+31.30%	+30.32%

5.4 Zero-Shot Generalization

Table ?? reports ADP reduction on all six held-out benchmarks (Table ??), for both seeds of the 11-benchmark policy, evaluated with zero benchmark-specific training. Two of the six (softmax, reduction_layer) were part of the training-time *universe* used to size the policy’s tensors (Section ??) but never sampled during training; the other four were entirely outside the universe as well, a strictly harder generalization test. Every benchmark finishes positive under both seeds — no sign flips, unlike the in-pool result in Section ?? — and the aggregate average is close across seeds (31.30% vs. 30.32%). Per-benchmark swings are real, however: custom_macbuf drops 13.4 points (61.2%→47.8%) and arm_core drops 5.1 points (12.1%→7.0%) from seed 42 to seed 7, partially offset by reduction_layer and mkDelayWorker32B moving the other way. We read this as evidence that the 11-benchmark policy’s zero-shot generalization is reasonably seed-stable in aggregate, but that any single benchmark’s zero-shot number should be read with a roughly ± 10 -point error bar until checked against a second seed, not treated as exact.

5.5 Fine-Tuning Efficiency

Separately from cross-benchmark transfer, we characterize how efficiently a multi-benchmark policy continues learning on a benchmark it has already partially seen during joint training (diffeq1, within the 11-benchmark pool), compared to training an identical architecture from scratch on that benchmark alone, both for 6,000 episodes under identical hyperparameters and separate VTR caches. The fine-tuned run dominates typical reward almost throughout (0.81 vs. 0.65 in the final 200-episode window, still climbing for scratch) and reaches its convergence plateau by roughly episode 1,600–1,800 (≈ 7 minutes wall-clock), after which it barely moves; scratch never plateaus within its full 6,000-episode budget. The mechanism is policy entropy collapse: the fine-tuned run’s entropy falls from 2.00 to 0.79 over the run (vs. scratch’s 4.63 to 2.42), so only 31.5% of its episodes are genuinely novel VTR evaluations versus 94.5% for scratch — it converges to a good region and stops exploring sooner, which is a sample-efficiency win here but, as Section ?? discusses, the same mechanism caps how much further exploration a fine-tuned policy is willing to do beyond its pre-trained prior. We note this result characterizes *continued learning* on a partially-known benchmark, not cross-benchmark transfer; the latter is addressed in Section ??.

5.6 Seed Variance: In-Pool vs. Zero-Shot

The two seeds of the 11-benchmark policy give a useful contrast between in-pool and zero-shot reliability. In-pool (Section ??), the pooled aggregate swings substantially across seeds (35.73% vs. 23.11%), driven almost entirely by one benchmark, or1200, finishing at 33.5% under seed 42 but only 5.8% under seed 7 — apparently a benchmark-specific convergence difference rather than a general property of the seed. Zero-shot (Section ??), by contrast, is far more stable in aggregate (31.30% vs. 30.32%) despite real per-benchmark movement, and critically shows no sign flips at all. We do not yet have an explanation for why in-pool fitting is more seed-sensitive here than zero-shot transfer is — it could be that in-pool performance is more exposed to which benchmarks a given seed’s policy happens to converge on slowly within a fixed episode budget (Section ??’s entropy-collapse mechanism plausibly compounds this: a seed that converges early on most benchmarks has fewer remaining episodes left for a slow benchmark like or1200 to catch up), while zero-shot performance reflects a more general property of the learned representation that is less sensitive to any one benchmark’s individual convergence trajectory. This is a characterization, not yet a tested hypothesis, and the most direct follow-up is a third seed to see which of the two patterns — the in-pool swing or the zero-shot stability — is the more typical one.

6 Conclusion

We asked whether a learned policy can do better than restarting evolutionary search from scratch for every new protected IP block in custom eFPGA-based logic masking. Under a fully controlled, apples-to-apples comparison — our own genetic algorithm and our RL policy on the identical benchmark, evaluation oracle, ADP metric, and traditional baseline — RL wins: 74.06% ADP reduction versus the GA’s 68.17% (Section ??). Jointly trained across 11 benchmarks, the same policy reaches a 35.73% aggregate in-pool ADP reduction with 10 of 11 benchmarks finishing positive (Section ??). Recomputed on GOLDS’ own published metric and baseline architecture, a genuinely zero-shot evaluation — no training on the target benchmark at all — reaches 37.4% versus GOLDS’ published $\sim 35\%$ (Section ??).

We also reported a result that complicates a clean amortization story: a second training seed shows that in-pool fitting can be considerably more seed-sensitive than zero-shot transfer is (Section ??) — the in-pool aggregate swings from 35.73% to 23.11% across seeds, driven by one benchmark, while the six-benchmark zero-shot aggregate stays close (31.30% vs. 30.32%) despite real per-benchmark movement (Section ??). We see this not as a weakness to be written around but as the most important finding for anyone trying to reproduce or build on this line of work — a single-seed result, in either direction, should not be taken as a stable characterization of the policy’s behavior until checked against at least a second seed.

Several gaps remain. Every result in this paper comes from one or two seeds, not the seed-averaged statistics with error bars a fully rigorous claim would need — itself a direct consequence of the variance finding above, and the most immediate item for any follow-up. The action space we search covers DSP/BRAM placement and aspect ratio only; the NSGA-II follow-up [?] additionally co-evolves CLB LUT size and custom arithmetic-block sizing, and extending

our formulation to that larger space is a natural next step, now that the GCN-based representation (Section ??) already generalizes across netlists of different size and resource mix. Our benchmark pool, while including hand-written probes specifically designed to stress DSP/BRAM combinations absent from real circuits, does

not yet cover the full set GOLDS and the NSGA-II follow-up report on (e.g. sha, the Koios deep-learning suite); broadening coverage would let every comparison in this paper be made directly rather than through the single-benchmark, metric-converted comparison in Section ??.

Temporary page!

L^AT_EX was unable to guess the total number of pages correctly. As there was some unprocessed data that should have been added to the final page this extra page has been added to receive it.

If you rerun the document (without altering it) this surplus page will go away, because L^AT_EX now knows how many pages to expect for this document.